

Braille Book: A Low-Cost Electromechanical Refreshable Braille Display

Hans Wassermann, Timothy Linker, Maria George, Arpan Sheth, Ayaan Uddin, Aman Grandhi, Saket Pabba, and Baslan Asaad

Abstract—An estimated 250 million people worldwide live with visual impairment. However, the use of Braille is severely limited, and existing refreshable Braille devices cost well over \$1,000, making them inaccessible to the majority of the population. That is where our project, Braille Book, comes in. We made an inexpensive electromechanical braille reader that generates characters using two rotating octagons for each braille cell, driven by stepper motors. A microcontroller-based Python pipeline translates English text into Grade 1 Braille using an open-source API. This pipeline maps each character to a motor state and transmits the target positions to an Arduino Uno, which calculates the minimum number of motor rotations required to display the correct braille cell. Our project is a two-cell unit with successful text-to-braille rendering, costing only \$37 to produce, and a scalable 20-cell device is estimated to cost only \$320, resulting in approximately an 80% cost reduction compared to current solutions. These results establish the viability of a low-cost, modular braille reader, and they lay the foundation for broader, more accessible braille devices for the visually impaired community.

CONTENTS

I	Introduction	1
I-A	Digital Element	1
I-B	Physical Prototype Element	2
I-C	Social Relevance and Application Element	2
II	Equipment and Methodology	2
II-A	Mechanical Design	2
II-A1	Octagon Geometry and Encoding Methodology	2
II-A2	Prototype 1: Direct Motor Shaft Drive	3
II-A3	Prototype 2: Bevel Gear Transmission	3
II-A4	Prototype 3: Spur Gear Transmission	3
II-B	Electronics	4
II-C	Software Architecture	5
II-C1	Braille Encoding	5
II-C2	Serial Communication	6
II-C3	Motor Control	6
III	Results and Discussion	6
III-A	Results	6
III-B	Cost Analysis	6
III-C	Future Work	7
IV	Conclusions	7
	References	7

I. INTRODUCTION

OVER 250 million people worldwide live with visual impairments, and current braille devices and textbooks cost between \$1,500 and \$15,000 or more. While braille literacy is crucial, the high cost of these devices makes them inaccessible to most people. The gap between the importance of braille literacy and the inaccessibility of existing devices inspired the development of Braille Book, a more accessible and inexpensive solution. The goal of this project is to design a low-cost prototype of an electromechanical braille reader that can dynamically display braille characters without using fixed pages. Through the use of 3D printed housing and gears driven by stepper motors and a software pipeline that efficiently converts English text to Grade 1 Braille, this work provides a proof of concept through a two-cell unit and demonstrates how it can be scaled to a 20-cell system with a dramatic price reduction compared to anything currently on the market.

Braille is a critical gateway to independence and literacy for people with visual impairments. Studies have demonstrated that children who learned Braille were more likely to be employed and to pursue higher education than those who were Braille illiterate [1]. In addition to better career outcomes, students proficient in braille reading perform better academically than those using only audio-based technologies, as audio-based education affords less direct interaction with written language. The primary barrier to braille adoption is cost: approximately 89% of the roughly 250 million visually impaired people worldwide live in regions characterized by poverty [2]. Furthermore, traditional printed braille books are large, difficult to transport, costly to produce, and unsuitable for everyday dynamic reading.

A. Digital Element

The software and electronics of this project are essential in updating the displayed braille characters. Three primary electronic systems are employed: a local computer, an Arduino microcontroller, and stepper motors. The local computer handles English-to-Braille conversion using an open-source library to translate English words and characters into their braille representations. A custom mapping correlates the resulting braille symbols to the correct motor face positions. These motor face states are then transmitted as a serial data stream to the Arduino microcontroller, which stores the current motor states and calculates the number of steps required to rotate each motor from its current position to the target face. An Arduino stepper motor library is used to send the relevant control signals to the motors. By encapsulating all conversion

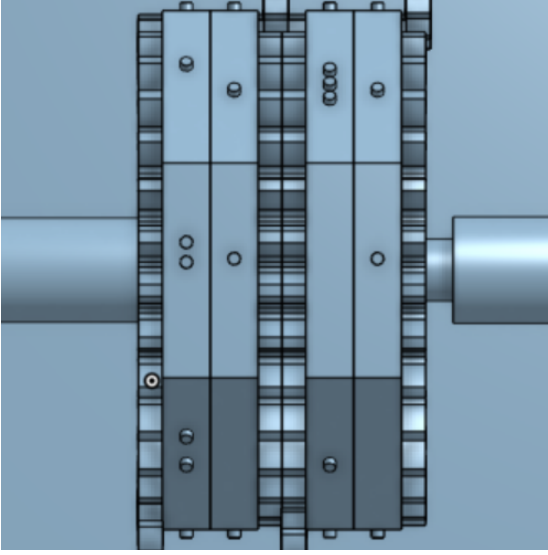


Fig. 1: Top view of adjacent octagon faces representing the physical braille output.

logic within software, this system provides an affordable mechanism for rendering electronic text as physical braille output.

B. Physical Prototype Element

The physical prototype allows the user to read a predetermined number of braille characters sequentially before all characters are refreshed to the next set. The prototype comprises octagonal prism structures mounted on a central axis in pairs, where each pair of octagons together represents a single complete braille character. The octagons present eight unique faces, such that rotating each prism changes which braille character the pair represents. Each octagon is independently motor-controlled through a dedicated gear-and-motor system, enabling precise and rapid rotary actuation. The user runs their hands along the top faces of the adjacent octagons in order to read the braille. Fig. 1 shows a top view of the adjacent octagon faces that the user traces from left to right above the central rod in order to read a string of braille characters. The number of braille characters displayed at a time directly corresponds to the number of octagon pairs. In the final prototype, two octagon pairs are implemented, enabling two braille characters to be represented simultaneously. This number is readily scalable by increasing the number of octagon pair assemblies mounted on the central rod.

C. Social Relevance and Application Element

Braille remains a tactile system that continues to support millions of people around the world in reading and writing [3]. Research conducted with legally blind adults has demonstrated that braille literacy is uniquely associated with well-being and self-esteem, stemming from its role as a critical pathway to information access for those unable to read print efficiently. Individuals who are braille literate demonstrate greater competence in participating in social and professional

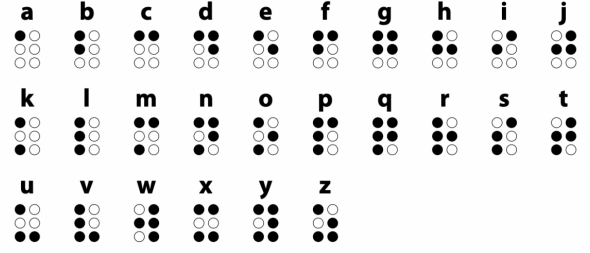


Fig. 2: Grade 1 Braille Alphabet.

activities. Furthermore, secondary braille readers have been shown to outperform non-braille readers across multiple outcomes, including employment [4]. These findings underscore the imperative to make braille more accessible for the visually impaired.

An electronic refreshable braille device increases convenience and independence while simultaneously reducing the costs and material waste associated with printing braille. Traditional braille books are expensive, bulky, and impractical for daily use. A dynamic braille display eliminates the need for large physical book collections, creating a cost-effective and portable alternative. Market research indicates growing demand for assistive technology, with the global braille device market projected to expand due to increasing accessibility regulations and educational inclusivity efforts. Public institutions, workplaces, and navigation systems could all benefit from a real-time braille text display that requires no costly physical printing. By making information more broadly accessible, this project aims to contribute directly to a more inclusive society.

II. EQUIPMENT AND METHODOLOGY

A. Mechanical Design

1) *Octagon Geometry and Encoding Methodology:* The core mechanical concept of the refreshable braille display is centered on an octagonal prism geometry, selected for its direct correspondence with the Grade 1 Braille standard. In Grade 1 Braille, each character is encoded as a 3×2 grid of raised dots, which can be decomposed into two independent columns of three dots each, as shown in Fig. 2. Each column of three dots admits $2^3 = 8$ distinct binary combinations, making a regular octagon a natural mechanical encoding structure. By assigning one unique dot combination to each of the eight faces, a single octagonal prism encodes every possible state of one braille column. Two such prisms positioned adjacently therefore represent a complete braille character across all 64 possible Grade 1 configurations.

Each octagonal prism face is fabricated with raised dot bumps in positions corresponding to its assigned braille column combination. The top face presented to the user's finger at any given time is determined by the rotational position driven by the stepper motor. The dot geometry and face assignments were established prior to fabrication and encoded directly into the motor control mapping in software, ensuring consistent correspondence between motor position and braille output.

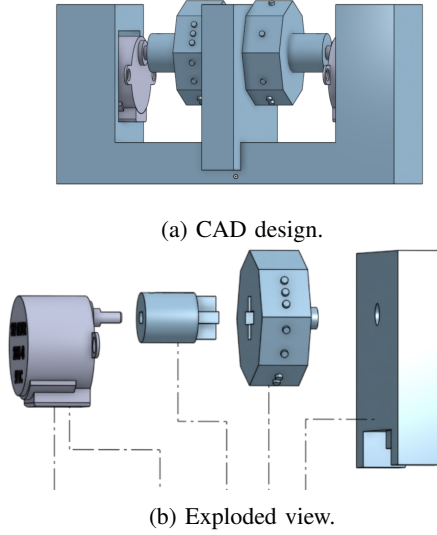


Fig. 3: First prototype CAD design and exploded motor-octagon connection.

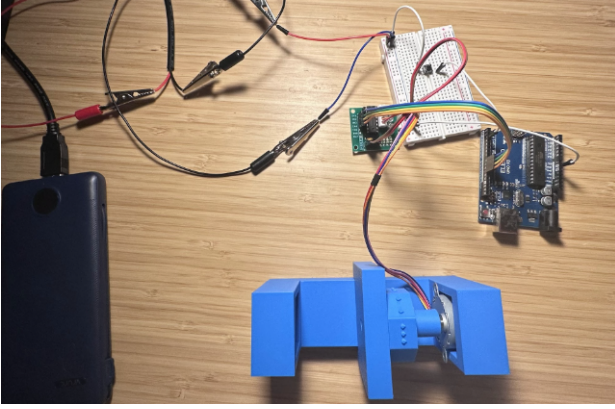


Fig. 4: First 3D printed prototype testing layout with one motor attached.

2) *Prototype 1: Direct Motor Shaft Drive*: The first prototype employed the most mechanically direct actuation method: the output shaft of each stepper motor was inserted directly into the center bore of its corresponding octagonal prism, driving rotation through a press-fit coupling. The assembly was housed within a 3D printed U-shaped enclosure in which the motor bodies were recessed into the side walls, with their shafts protruding inward to engage the prisms, as shown in Fig. 3a. A central dividing wall provided axial constraint, serving as a bearing surface against which each prism rotated, as illustrated in the exploded view in Fig. 3b. The physical testing layout is shown in Fig. 4.

While mechanically simple, this configuration presented fundamental scalability limitations. The motor body occupied the full lateral envelope adjacent to each octagonal prism, making it infeasible to position additional octagon pairs within a spacing compatible with legible multi-character braille output. Despite these limitations, the prototype was sufficient to support concurrent software development while mechanical

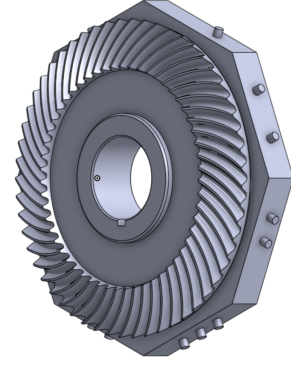


Fig. 5: Bevel gear octagon design, showing gear inset from outer perimeter.

iteration continued.

3) *Prototype 2: Bevel Gear Transmission*: The primary design objective of the second prototype was to reduce the inter-octagon spacing within a single braille cell, as excessive separation between the two columns of a character would significantly compromise tactile readability. To achieve this, a bevel gear transmission was introduced, reorienting the motor shaft perpendicular to the rotational axis of the octagonal prism rather than coaxial with it. A bevel gear was integrated directly onto the larger flat end face of each prism, on the face opposite the braille dot surfaces. As shown in Fig. 5, this gear was intentionally inset from the outer perimeter of the octagonal cross-section so that the gear teeth would remain imperceptible to a user's finger during braille reading. A corresponding bevel gear adapter was designed to engage the motor shaft through a press-fit double-D bore, meshing with the octagon-mounted gear to transmit rotation through a 90-degree shaft angle. The housing was redesigned so that the motors were embedded vertically within the base of the U-shaped enclosure, allowing the two octagonal prisms of each braille cell to be positioned in close proximity, as shown in Fig. 6a and Fig. 6b. The physical prototype and motor bevel gear adapter are shown in Fig. 7.

This configuration represented a substantial improvement in cell compactness relative to the first prototype. However, scaling to multiple characters revealed a critical geometric constraint: the outer diameter of the motor-side bevel gear adapter established a minimum required separation between adjacent braille cells, as any reduction in cell pitch below this diameter would result in mechanical interference between the motor gear and the neighboring octagon assembly. Attempts to reduce the motor bevel gear diameter were constrained by the resolution of the fused deposition modeling (FDM) printer employed, which could not reliably produce functional gear tooth geometry below a threshold tooth height using polylactic acid (PLA) filament. While this transmission approach would be viable under higher-precision manufacturing conditions, it was not achievable within the constraints of the available fabrication equipment.

4) *Prototype 3: Spur Gear Transmission*: The third and final prototype retained the fundamental design principle of the second iteration, namely offsetting the motor from the octag-

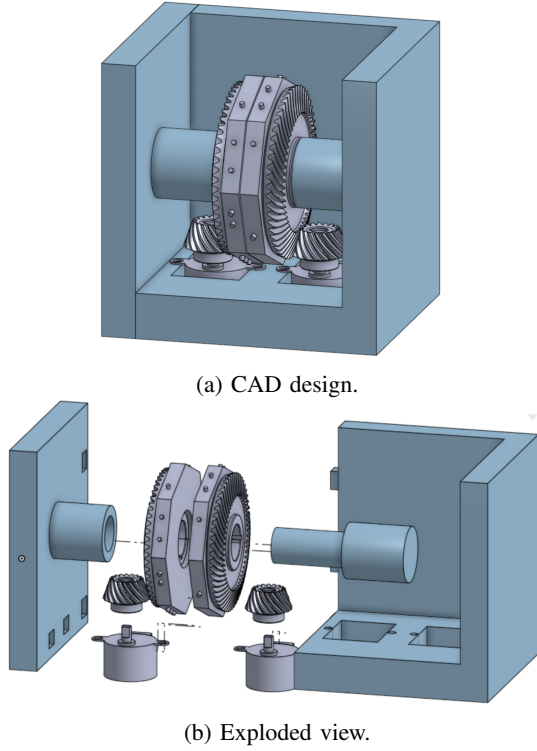


Fig. 6: Second prototype CAD design and exploded bevel gear assembly.

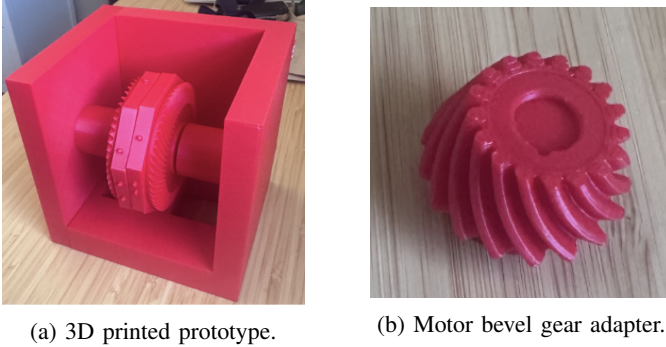


Fig. 7: Second prototype physical build and motor bevel gear adapter.

onal prism's rotational axis to minimize inter-prism spacing, while replacing the bevel gear transmission with a spur gear system. A worm gear configuration was evaluated during the design process but was rejected due to the characteristically high gear ratios associated with worm drives, which would have produced unacceptably low rotational velocities at the octagon given the step rate of the stepper motors employed. The spur gear approach requires the motor shaft axis to be parallel to the octagonal prism's rotational axis, as both gears must operate within a common rotational plane for proper tooth engagement. Consistent with the preceding prototype, the octagon-mounted spur gear is inset from the outer perimeter of the prism faces to ensure that the gear teeth are not perceptible to a user's finger during tactile reading, as shown

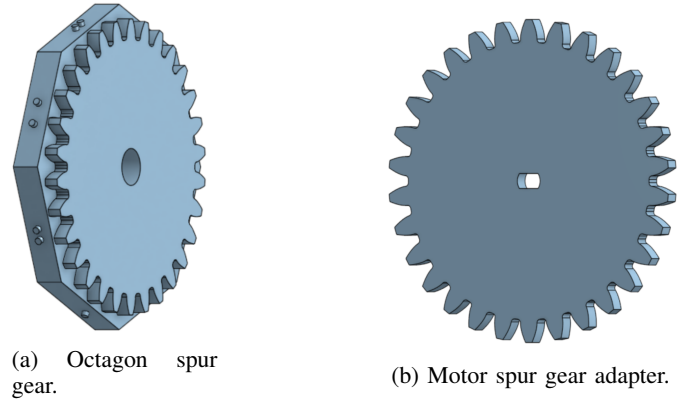


Fig. 8: Third prototype spur gear components.

in Fig. 8a. A motor-side spur gear was designed with a double-D center bore dimensioned to interface directly with the motor output shaft, as shown in Fig. 8b. Both gears were minimized to 3 mm in thickness, as this represented the minimum face width at which functional PLA-printed spur gear teeth could be reliably produced while still maintaining sufficient structural integrity to transmit the required torque.

Given that the microcontroller employed provided sufficient general-purpose input/output (GPIO) pins to drive four stepper motors simultaneously, the final housing was designed to accommodate two complete octagon pairs, enabling the simultaneous display of two braille characters. The U-shaped housing architecture of the prior prototypes was redesigned as a two-component system comprising an L-shaped primary body and a separately attachable end wall. The L-shaped body incorporates a cylindrical bore on its upper face through which a stepped cylinder is inserted. The two-diameter stepped geometry of the cylinder prevents axial translation of the octagonal prisms along the cylinder axis. During assembly, the prisms are loaded sequentially onto the cylinder, after which the removable end wall is pressed into position to fully constrain the assembly, as shown in Fig. 9a and Fig. 9b. Intentional dimensional clearances were incorporated into the housing geometry to prevent excessive compressive contact between adjacent prism faces, ensuring each octagonal prism rotates freely without frictional interference from neighboring components. A 2 mm thick rubber washer was inserted between the two octagon pairs to prevent the motor spur gear teeth from simultaneously meshing with both adjacent octagon gears, as shown in Fig. 10.

B. Electronics

The octagonal prisms are actuated using 28BYJ-48 stepper motors, shown in Fig. 11a. Stepper motors were selected over servo motors because they permit precise incremental rotation to any position across a full 360-degree range, whereas standard hobby servos are limited to 180-degree travel and do not natively support discrete positional step control. To drive the 28BYJ-48 motors, ULN2003 motor driver boards were employed, as shown in Fig. 11b. Motor driver boards are necessary to translate low-current control signals from

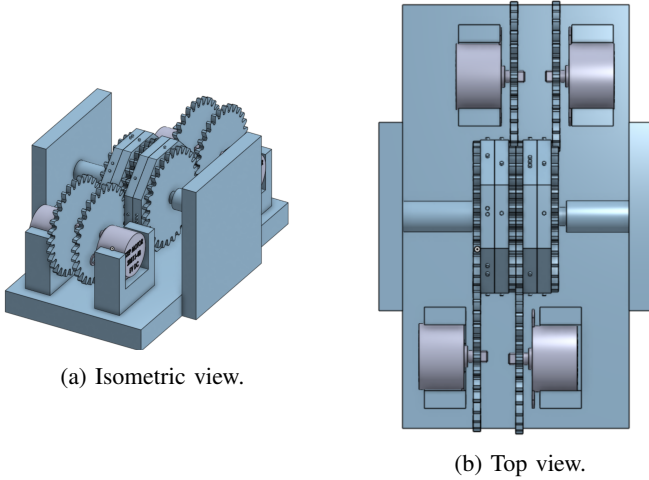


Fig. 9: Final prototype CAD assembly views.

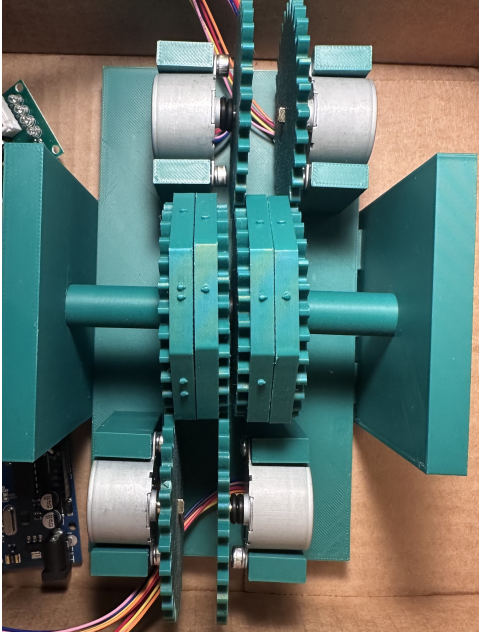


Fig. 10: Rubber washer spacer between octagon pairs in final assembly.

the microcontroller into the higher-current switching signals required to energize the motor coil phases. The driver board also electrically isolates the microcontroller from the motor power supply, protecting against elevated currents and voltage transients that would otherwise damage the microcontroller's digital output pins.

An Arduino Uno R3, shown in Fig. 12, was selected as the microcontroller for this project due to its familiarity within the team, its low cost, and its native compatibility with the ULN2003 motor driver boards. The 28BYJ-48 stepper motors cannot be powered directly from the Arduino, as the microcontroller imposes an inherent current limit of 40 mA per digital output pin, which is substantially lower than the current required to drive the motor coils. To address this constraint, a battery pack in conjunction with a USB to bare wire power

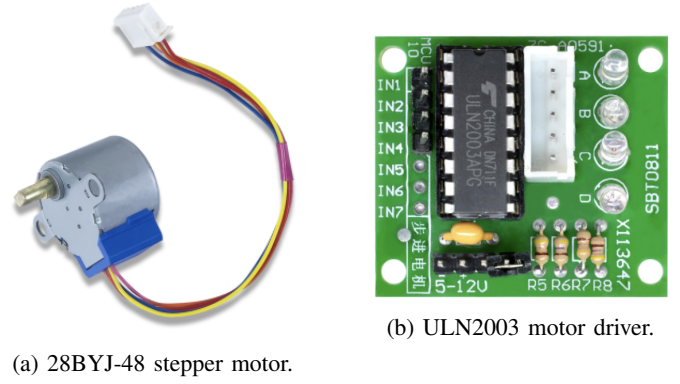


Fig. 11: Actuation components: stepper motor and driver board.



Fig. 12: Arduino Uno R3 microcontroller.

cord was used to supply independent power to the motors, decoupling motor power from the Arduino's regulated output.

C. Software Architecture

1) *Braille Encoding*: The first stage of the software pipeline ingests the characters to be translated into braille. User input is captured via a Python `input()` function, constrained to the two-character capacity of the hardware:

```
wordSizeLimit = 2
user_input = input(
    f"Enter_a_word_with_a_limit_of_"
    f"{wordSizeLimit}_character(s):_")
)
```

The input string is then translated into braille using `english2braille-api`, an open-source REST API implemented in Flask [5]. An API request is submitted with the input word, and the corresponding braille representation is returned. The primary challenge encountered was converting the unicode braille output of the API (e.g. `⠠⠠`) into a format interpretable by the Arduino motor controller. To accomplish this, a conversion pipeline was developed that first transforms the unicode output into two columns, each represented as a 3×1 binary matrix corresponding to the two motors of a braille character. Each element of the matrix contains a 1 if the corresponding dot is raised and a 0 if it is not. The binary column vectors are then mapped to integer face indices using two hashmaps defined as follows:

```

motor1_map = {
  "000": 0, "111": 1, "011": 2,
  "101": 3, "110": 4, "100": 5,
  "010": 6, "001": 7
}
motor2_map = {
  "000": 0, "100": 1, "010": 2,
  "001": 3, "011": 4, "101": 5,
  "110": 6, "111": 7
}

```

The binary column matrices are converted to strings and passed through the corresponding hashmaps to produce integer target face indices, which are subsequently transmitted to the Arduino.

2) *Serial Communication*: The `pySerial` library is used to establish a serial connection between the Arduino microcontroller and the host computer. The connection is initialized as follows:

```

arduino = serial.Serial('COM3', 9600, timeout
=2)

```

Data is written to the Arduino using the library's `write()` function:

```

arduino.write(command.encode())

```

Diagnostic output from the microcontroller can be retrieved using the library's `readline()` function:

```

line = arduino.readline()

```

Serial communication was selected due to its native support within the Arduino development environment, its reliability, and its low latency, which is critical for a system that requires precise synchronization between character rendering and motor actuation. The serial connection is bidirectional, enabling the host computer to both write target face states and read diagnostic output or error messages from the microcontroller. The Arduino serial buffer preserves the ordering of face state commands when multiple characters are transmitted sequentially.

3) *Motor Control*: The Arduino firmware receives face state indices over serial and implements the logic required to rotate each octagonal prism to the corresponding target face. Four stepper motors are initialized, and a `spinToFace()` function computes the minimum rotation path from the current face to the target face, selecting either the forward or reverse direction based on which requires fewer steps. The 28BYJ-48 stepper motor requires 2048 steps per full revolution; with eight faces per octagon, each face transition corresponds to 256 steps. The motors are actuated sequentially: the left motor of the first character rotates before the right, and this sequence repeats for the second character. The core motor control function is implemented as follows:

```

void spinToFace(Stepper &wheel, int &current,
               int target, bool isRight) {
  if (current == target) return;
  int forward =
    (target - current + TOTAL_FACES) %
    TOTAL_FACES;
  int backward = TOTAL_FACES - forward;
  if (forward <= backward) {
    if (isRight) { forward = -forward; }
    wheel.step(forward * STEPS_PER_FACE);
  } else {
    if (isRight) { backward = -backward; }
    wheel.step(-backward * STEPS_PER_FACE);
  }
  current = target;
}

```

III. RESULTS AND DISCUSSION

A. Results

The final physical prototype successfully translated two English characters into their corresponding braille representations by rotating each stepper motor to the correct face position for every octagonal prism in the assembly. During final assembly and testing, several mechanical issues were identified that would benefit from remediation in future iterations.

The first issue concerned octagon alignment within the housing. As discussed in Section II-A, a rubber washer was inserted between the two octagon pairs to prevent the motor spur gear teeth from meshing with both adjacent octagon gears simultaneously. However, because the motor mounting positions are fixed within the housing, each octagon spur gear must be precisely aligned with its corresponding motor spur gear for proper tooth engagement. Due to insufficient dimensional tolerances in the housing geometry, some octagonal prisms were not seated perfectly perpendicular to the cylinder axis, and instead exhibited a slight angular offset in the direction of the meshing force applied by the motor spur gear. This misalignment compromised the consistency of face presentation across the assembly.

The second issue concerned gear backlash. During testing, a small but measurable amount of rotational play was observed in the motor spur gears, which could be displaced laterally without actuating the motor shaft. This backlash introduced positional inaccuracies when rotating the octagonal prisms to their target faces, which is particularly consequential for braille readability given the precise spatial requirements of raised dot positioning.

A third contributing factor to misalignment was hypothesized to be frictional interference between adjacent octagonal prisms during rotation. It is possible that contact forces between neighboring prism faces impeded the motor from rotating each octagon to its exact target position. This effect may have been compounded by variability in the internal mechanical characteristics of individual stepper motors, though definitive attribution would require additional iterative testing to isolate each variable.

B. Cost Analysis

The bill of materials for the two-cell prototype and the projected cost for a 20-cell system are summarized in Table I.

TABLE I: Bill of Materials and Cost Projection

Component	Unit Cost	Qty	2-Cell	20-Cell
28BYJ-48 Motor	\$3	4	\$12	\$120
Arduino Uno R3	\$15	1	\$15	\$50
Printed Materials	–	–	≤\$10	≤\$150
Total	–	–	\$37	\$320

The two-cell prototype was produced at a total cost of \$37. Projecting this design to a 20-cell configuration yields an estimated cost of \$320, representing approximately an 80% reduction relative to commercially available 20-cell refreshable braille displays, which typically retail in the range of \$3,000 or more. This cost differential demonstrates that the proposed approach achieves the primary project objective of producing a substantially more affordable alternative to existing solutions.

C. Future Work

Several improvements are planned to advance Braille Book from a proof-of-concept prototype toward a more compact, reliable, and scalable device. Mechanically, alternative gear configurations will continue to be explored. While the current spur gear design achieves adequate inter-octagon spacing, worm gears represent a promising direction for future iterations, as they would provide a more compact layout and a more secure rotational lock between face positions, improving consistency of alignment. The characteristically high gear ratios of worm drives would necessitate the adoption of faster, more powerful motors to maintain acceptable face transition speeds. An additional mechanical improvement would be the integration of a physical alignment mechanism, such as detents or snap features, to ensure each octagonal prism consistently arrests at the correct face position, mitigating the drift and calibration error observed during testing of the final prototype.

On the electronics side, the primary goal is to improve system modularity to simplify repair and facilitate scaling. The target configuration is a 20-cell or greater display capable of rendering full words and phrases. As the motor count increases, direct wiring from each motor to the Arduino becomes impractical; multiplexers or dedicated motor controller integrated circuits would be required to manage the expanded input/output requirements.

On the software side, a mechanism for loading and advancing through text files stored directly on the device would

significantly improve usability, eliminating the dependency on a connected computer and enabling users to navigate content using a simple button interface. Through these developments, Braille Book would advance from a working proof-of-concept to a portable, reliable, and practical low-cost refreshable braille reader suitable for broader deployment.

IV. CONCLUSIONS

Braille Book is an inexpensive refreshable braille reader that displays braille characters using two octagonal prisms per character, actuated by stepper motors and coordinated through an Arduino-based control system and a Python translation pipeline. With the two-cell prototype costing \$37 to produce and a projected 20-cell device estimated at \$320, the system is significantly more affordable than existing commercial alternatives, which retail at approximately \$3,000 for comparable cell counts. Through this cost reduction, the millions of visually impaired individuals who could not previously afford a refreshable braille display would gain access to dynamic digital text, opening educational, professional, and social opportunities that braille literacy is known to enable. This project demonstrates the viability of a low-cost, modular, and scalable approach to refreshable braille display design, and establishes a foundation for future development toward a fully functional multi-cell device.

ACKNOWLEDGEMENTS

We would like to thank Dean Antoine, Megan Holleran, our design advisor Noa Strupinsky, our alumni mentor Cecilia Diaz, and the rest of the design advisor team for guiding this project to where it is today.

REFERENCES

- [1] R. Ryles, “The impact of braille reading skills on employment, income, education, and reading habits,” *Journal of Visual Impairment & Blindness*, vol. 90, no. 3, pp. 219–226, 1996.
- [2] A. Haileamlak, “Visual impairment and blindness,” *Ethiopian Journal of Health Sciences*, vol. 32, no. 1, pp. 1–2, 2022.
- [3] NECO, “The importance of braille literacy,” 2025. [Online]. Available: <https://neco.org>
- [4] A. M. Silverman and J. L. Bell, “The association between braille reading history and well-being for blind adults,” *Journal of Blindness Innovation and Research*, vol. 8, no. 1, 2018.
- [5] “english2braille-api,” GitHub. [Online]. Available: [https://github.com/\[repo-link\]](https://github.com/[repo-link])